

# SOUTHEAST UNIVERSITY



*Meeting the Challenges of Time*

## PROJECT REPORT ON LIBRARY MANAGEMENT SYSTEM

### Submitted By

1.Name: PUJA RANI PAUL

ID : 2024200000447

2.Name: NAZIA TAZKIA

ID : 2024200000460

3.Name: THAMINA ISLAM TENNI

ID : 2024200000617

Course Code: CSE282.6

Course Title : Introduction to Programming Language II  
(Java)

Github Link : [GitHub - Library Management System](#)

Dept. of Computer Science and Engineering, SEU

### Submitted to:

Name: Md. Ayon Mia

Designation: Lecturer

Dept. of Computer Science and Engineering, SEU

**Date of Submission: 28.092026**

# TABLE OF CONTENTS

|                                                     |                |
|-----------------------------------------------------|----------------|
| <b>1.Introduction .....</b>                         | <b>Page 3</b>  |
| <b>2.Objectives .....</b>                           | <b>Page 3</b>  |
| <b>3.Motivation .....</b>                           | <b>Page 3</b>  |
| <b>4.Applications .....</b>                         | <b>Page 3</b>  |
| <b>5.Project Description .....</b>                  | <b>Page 4</b>  |
| ● 5.1System Overview .....                          | Page4          |
| ● 5.2Major Classes & OOP Concepts .....             | Page4          |
| ● 5.3System Workflow.....                           | Page5          |
| <b>6.Tools &amp; Technologies .....</b>             | <b>Page 5</b>  |
| <b>7.GUI Screenshots &amp; Output/Results .....</b> | <b>Page 6</b>  |
| <b>8.Contribution .....</b>                         | <b>Page 11</b> |
| <b>9.Conclusion .....</b>                           | <b>Page 15</b> |

## 1.Introduction

The **Library Management System** is a Java-based desktop application designed to automate the manual operations of an academic or public library. The system replaces traditional paper-based tracking with a dynamic, event-driven Graphical User Interface (GUI). It enables seamless registration of library items (Books, Magazines), member profile tracking, automated checkout (issuing), returning of items, and dynamic catalog searching with local file persistence.

## 2. Objectives

**Aims:** To replace manual paper-based record-keeping with an efficient, error-free automated system.

### Specific Goals:

- Implement modular Java OOP principles (Abstraction, Inheritance, Polymorphism, Encapsulation).
- Build a GUI using Java Swing (JFrame, JPanel, CardLayout).
- Ensure data persistence using File I/O (Mylibrary\_data.txt).

**Key Functionalities:** Catalog viewing, item addition/deletion, item search, item issue, return processing, and custom exception handling.

## 3. Motivation

- **Why Selected:** Manual library management leads to lost records, stock miscalculations, and slow issue/return processing.
- **Real-World Need:** Libraries require real-time item tracking, validation rules (e.g., limit checkouts per member), and automatic persistent file saving.

## 4. Applications

- **Where Used:** University departments, school libraries, and small community reading spaces.
- **Potential Users:** Library Administrators, Librarians, Students, and Faculty Members.
- **Practical Benefits:** Speeds up transaction times, prevents duplicate entries, and keeps structured file backups.

## 5. Project Description

### ■ Main Features:

- **View Catalog:** Table interface displaying available items, copy counts, and attributes.
- **Issue & Return Item:** Handles checkout rules, calculates due dates, and updates inventory stock dynamically.
- **Update/Delete Item:** Allows editing title, total copies, author/publisher details, or removing obsolete items.

### ■ Major Classes & Architecture:

- **MainFrame:** Root GUI container using tabbed components for seamless navigation.
- **LibraryService:** Central business logic class for managing collections, search algorithms, and file saving.
- **Book & Magazine:** Subclasses inheriting common properties from LibraryItem.
- **Exception handling:** Custom exception classes ensure operational reliability and data consistency:

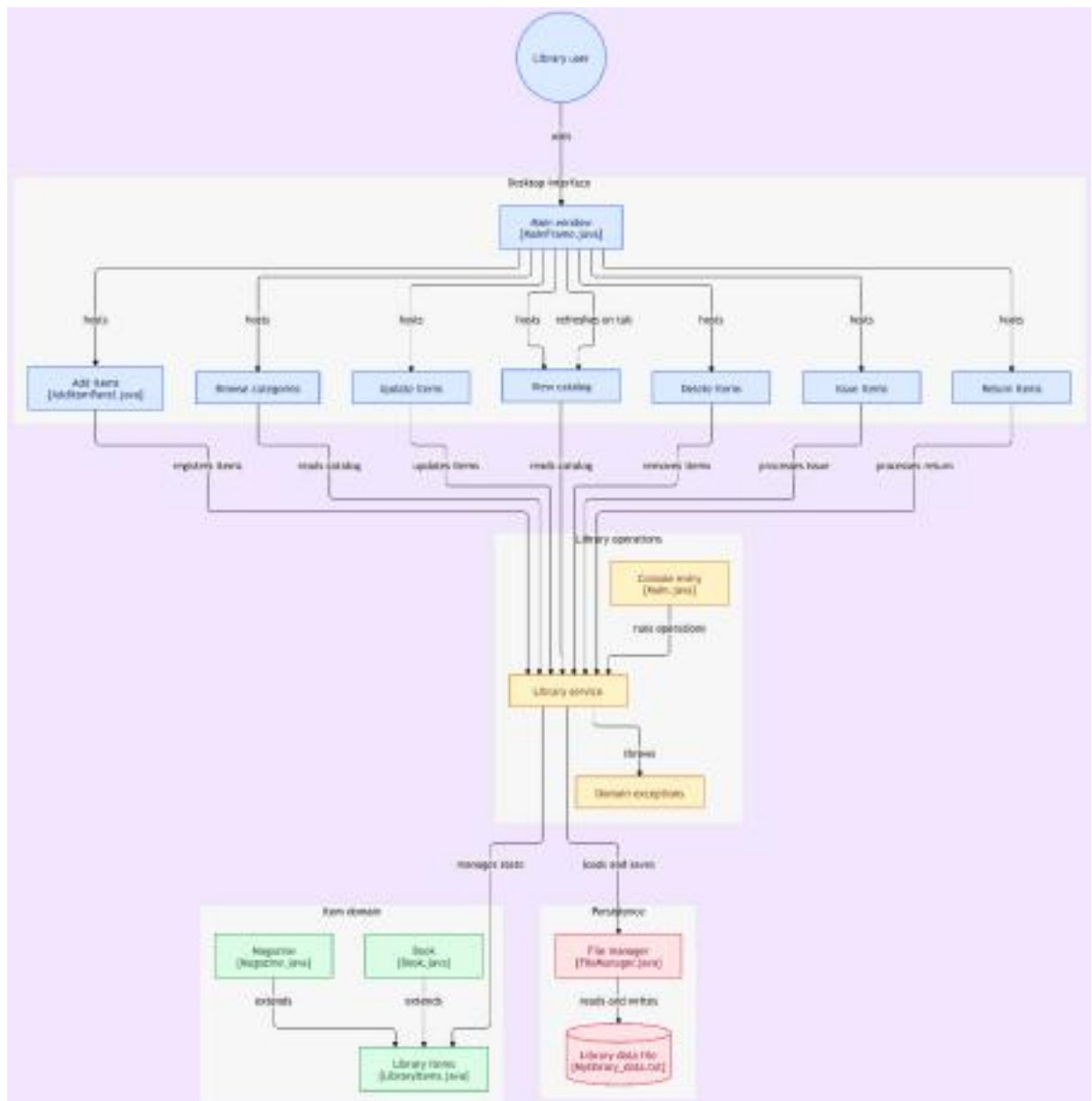
✧ **Services using Exceptions:** LibraryService

✧ **Custom Custom Exceptions:**

- ItemNotFoundException=Triggered when a search or checkout title does not exist.
- InvalidInputException= Triggered when numeric fields receive string inputs
- for robust user error prevention.

| OOP Concept          | How It's Used                                                                                                                                                                                                                                                                         |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Inheritance</b>   | <p>Book and Magazine extend the abstract class LibraryItem. Child classes extend the base abstract class to reuse core properties.</p> <pre> LibraryItem (Base Class) ├── Book └── Magazine           </pre>                                                                          |
| <b>Encapsulation</b> | <p>Class attributes are private and accessed through getters and setters</p> <p><b>Classes using Encapsulation:</b> LibraryItem, Book, Magazine, LibraryService.</p>                                                                                                                  |
| <b>Polymorphism</b>  | <p>Item descriptions use method overriding Implemented via method overriding where overridden getDetails() produces specialized details based on runtime object type.</p>                                                                                                             |
| <b>Abstraction</b>   | <p>LibraryItem is declared as an abstract class. The abstract method getDetails() hides internal implementation details. Complex file I/O and UI rendering are hidden behind modular GUI panels.</p> <p><b>Concrete Implementations:</b> Book.getDetails(), Magazine.getDetails()</p> |

## Diagram : System Workflow



## 6.TOOLS & TECHNOLOGIES

- **Programming Language:** Java (JDK 21/26)
- **IDE:** IntelliJ IDEA
- **GUI Framework:** Java Swing & AWT (JFrame, JPanel, CardLayout, GridBagLayout, BorderLayout, Custom BasicButtonUI)
- **Persistence:** Java File I/O (BufferedReader, BufferedWriter)
- **Version Control:** Git & GitHub

## 7. Gui ScreenShots & Output / Results

- **Main Dashboard / GUI Frame:** Displays the library system's main window and navigation tabs.

The screenshot shows the 'SOUTHEAST SMART LIBRARY' main dashboard. At the top, there's a header with the library name and a subtitle 'Manage Books, Magazines, Members and Library Records'. Below this is a navigation bar with tabs: 'Add Item', 'View Catalog', 'View Category', 'Update Item', 'Delete Item', 'Issue Item', and 'Return Item'. The 'Add Item' tab is active. The main content area is titled 'Add Library Item' with the subtitle 'Add a new book or magazine to the library'. It contains a form with four fields: 'Item Type' (a dropdown menu with 'Book' selected), 'Title', 'Total Copies', and 'Author'. Below the form are two buttons: 'Add Item' and 'Clear'. At the bottom, there's a tip: 'Tip: Enter all required information before adding the item. The record will be saved automatically.'

- **Add Item Panel:** Shows a success message after a book is added

This screenshot shows the same 'Add Library Item' form as the previous one, but with a success message dialog box overlaid. The dialog box is titled 'Success' and contains the message 'Book added successfully!'. It also displays the details of the added book: 'Title: Numerals Method', 'Author: Richard L. burden', and 'Total Copies: 2'. There is an 'OK' button at the bottom of the dialog box. The background form and navigation elements are the same as in the previous screenshot.

- **View Catalog view:** Displays all books and magazines with their details and available copies.

SOUTHEAST SMART LIBRARY

### SOUTHEAST SMART LIBRARY

Manage Books, Magazines, Members and Library Records

[Add Item](#)
[View Catalog](#)
[View Category](#)
[Update Item](#)
[Delete Item](#)
[Issue Item](#)
[Return Item](#)

#### Library Catalog

View all books and magazines available in the library

| Type     | Title                              | Author / Publisher  | Available Copies |
|----------|------------------------------------|---------------------|------------------|
| Book     | Advanced Java                      | Herbert Schildt     | 5                |
| Book     | Python Programming                 | Guido van Rossum    | 5                |
| Book     | C++ Programming                    | Bjarne Stroustrup   | 4                |
| Book     | Data Structures                    | Mark Allen Weiss    | 6                |
| Book     | Numerical Methods                  | Richard L. burden   | 8                |
| Book     | Data Structures & Algorithms       | Mark Allen Weiss    | 31               |
| Book     | Data Communications and networking | Behrouz a forouzan  | 19               |
| Magazine | Kishore Alo                        | Prothom Alo         | 12               |
| Magazine | Time Magazine                      | Time USA            | 7                |
| Magazine | Canvas                             | Canvas Publications | 21               |
| Magazine | Jingala Magazine                   | Mr. Jhandubam       | 9999             |
| Book     | 101 Ways to Fail in Exam           | Mr. Md CrowMan      | 11               |
| Magazine | Vogue                              | Conde Nast          | 1                |

- **View Category:** Filters library items by books or magazines.

SOUTHEAST SMART LIBRARY

### SOUTHEAST SMART LIBRARY

Manage Books, Magazines, Members and Library Records

[Add Item](#)
[View Catalog](#)
[View Category](#)
[Update Item](#)
[Delete Item](#)
[Issue Item](#)
[Return Item](#)

#### View Library by Category

Filter library items by Books or Magazines

View Category: Books Refresh

| Type | Title                              | Author / Publisher | Total Copies | Available Copies | Status    | Member ID |
|------|------------------------------------|--------------------|--------------|------------------|-----------|-----------|
| Book | Advanced Java                      | Herbert Schildt    | 5            | 5                | Available | N/A       |
| Book | Python Programming                 | Guido van Rossum   | 5            | 5                | Available | N/A       |
| Book | C++ Programming                    | Bjarne Stroustrup  | 4            | 4                | Available | N/A       |
| Book | Data Structures                    | Mark Allen Weiss   | 6            | 6                | Available | N/A       |
| Book | Numerical Methods                  | Richard L. burden  | 8            | 8                | Available | N/A       |
| Book | Data Structures & Algorithms       | Mark Allen Weiss   | 31           | 31               | Available | N/A       |
| Book | Data Communications and networking | Behrouz a forouzan | 19           | 19               | Available | N/A       |
| Book | 101 Ways to Fail in Exam           | Mr. Md CrowMan     | 11           | 11               | Available | N/A       |
| Book | Numerals Method                    | Richard L. burden  | 2            | 2                | Available | N/A       |

**Category Filter**  
Select a category to view specific library items.

- **Update item:** Confirms that the library item's details were updated successfully.

The screenshot shows the 'SOUTHEAST SMART LIBRARY' interface. The navigation bar includes 'Add Item', 'View Catalog', 'View Category', 'Update Item' (highlighted), 'Delete Item', 'Issue Item', and 'Return Item'. The main heading is 'Update Library Item' with the subtitle 'Edit the information of an existing library item'. On the left, there are input fields for 'Item Type' (set to 'Book'), 'Current Title', 'New Title', 'New Total Copies', and 'New Author'. A modal dialog titled 'Update Successful' is open, displaying an information icon and the message 'Book updated successfully!'. It lists the changes: 'Old Title: Numerical Methods', 'New Title: Numerical Method', 'Total Copies: 2', and 'Author: Richard L. burden'. The dialog has an 'OK' button. Below the dialog are 'Update Item' and 'Clear' buttons. At the bottom, a light blue box contains the text: 'Update Information. Enter the current title and the new information. The updated record will be saved automatically.'

- **Delete item:** Asks the user to confirm before deleting an item.

The screenshot shows the 'SOUTHEAST SMART LIBRARY' interface. The navigation bar includes 'Add Item', 'View Catalog', 'View Category', 'Update Item', 'Delete Item' (highlighted), 'Issue Item', and 'Return Item'. The main heading is 'Delete Library Item' with the subtitle 'Remove an item from the library'. A modal dialog titled 'Confirm Deletion' is open, displaying a warning icon and the message 'Are you sure you want to delete:'. It shows the item title 'Numerals Method' and states 'This action cannot be undone.' with 'Yes' and 'No' buttons. Below the dialog, a light red box contains a warning: 'Warning: Deleting an item cannot be undone. An item cannot be deleted while one of its copies is issued.'

- **Issue Item Form input:** Enters the item, member, and borrowing details.

The screenshot shows the 'Issue Library Item' form in the Southeast Smart Library system. The form is titled 'Issue Library Item' with the subtitle 'Issue a book or magazine to a library member'. It contains several input fields: 'Item Title' (Vogue), 'Member ID' (447), 'Member Email' (2024200000@gmail.com), 'Member Type' (Student), 'Issue Day' (7), and 'Quantity' (2). There are two buttons at the bottom: 'Issue Item' and 'Clear'. The form is set against a light blue background with a navigation bar at the top containing links like 'Add Item', 'View Catalog', 'View Category', 'Update Item', 'Delete Item', 'Issue Item', and 'Return Item'. A 'Borrowing Rules' section at the bottom states: 'Student: Maximum 3 items / 15 days' and 'Faculty: Maximum 5 items / 30 days'.

**Issue Error:** Shows a message because it is unavailable or the borrowing limit was reached.

This screenshot shows the same 'Issue Library Item' form as the previous one, but with an error message displayed. The error message is a dialog box titled 'Invalid Input' with a red 'X' icon. The text inside the dialog box reads: 'Process failed: Item cannot be issued. Check availability or borrowing limit.' There is an 'OK' button at the bottom of the dialog box. The form fields and buttons are still visible in the background.

- **Issue Item:** Confirms that the item was issued to the member.

**SOUTHEAST SMART LIBRARY**  
Manage Books, Magazines, Members and Library Records

Add Item View Catalog View Category Update Item Delete Item **Issue Item** Return Item

### Issue Library Item

Issue a book or magazine to a library member

Item Title

Member ID

Member En

Member Ty

Issue Day

Quantity

X

Issue Successful

Item issued successfully!

Item: Canvas

Member ID: 447

Member Type: Student

**Borrowing Rules**  
Student: Maximum 3 items / 15 days

- **Return item:** Confirms the item was returned and shows a late fine of **20 BDT per day** because the student kept it for more than the allowed **15 days**.

**SOUTHEAST SMART LIBRARY**  
Manage Books, Magazines, Members and Library Records

Add Item View Catalog View Category Update Item Delete Item Issue Item **Return Item**

### Return Library Item

Return an issued item and calculate late fine

Item Title

Return Date

X

Return Successful

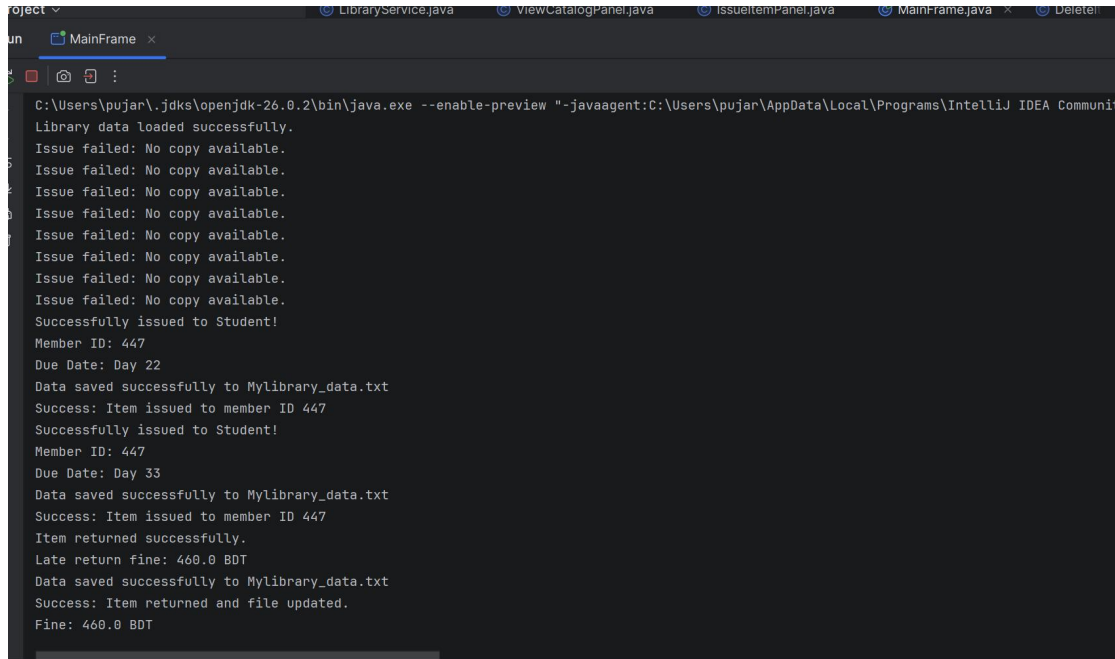
Item returned successfully!

Item: Canvas

Late Fine: 460.00 BDT

**Late Fine: 460.00 BDT**

## ● Output:



```
C:\Users\pujar\.jdk\openjdk-26.0.2\bin\java.exe --enable-preview "-javaagent:C:\Users\pujar\AppData\Local\Programs\IntelliJ IDEA Commun
Library data loaded successfully.
Issue failed: No copy available.
Issue failed: No copy available.
Issue failed: No copy available.
Issue failed: No copy available.
Issue failed: No copy available.
Issue failed: No copy available.
Issue failed: No copy available.
Issue failed: No copy available.
Issue failed: No copy available.
Successfully issued to Student!
Member ID: 447
Due Date: Day 22
Data saved successfully to Mylibrary_data.txt
Success: Item issued to member ID 447
Successfully issued to Student!
Member ID: 447
Due Date: Day 33
Data saved successfully to Mylibrary_data.txt
Success: Item issued to member ID 447
Item returned successfully.
Late return fine: 460.0 BDT
Data saved successfully to Mylibrary_data.txt
Success: Item returned and file updated.
Fine: 460.0 BDT
```

## 8. Contribution

**Contributor/Member 1:** Puja Rani Paul

**Module :** Project Design, Main Base Class, and System Improvements

### 1. Project Setup and Design

- **Repository and Development Setup:** I created the project repository on GitHub, set up Git in IntelliJ IDEA, and organized the project folders.
- **Main Base Class :** I created the abstract class LibraryItems.java. It provides the basic structure for library items such as Book and Magazine.
- **Documentation:** I wrote the main README.md file. It explains the system design, team responsibilities, and use of OOP concepts.

### 2. Use of OOP Concepts

- **Abstraction:** I made LibraryItems an abstract class and added the abstract method getDetails(). Child classes must provide their own version of this method.
- **Encapsulation:** I made the variables id, title, totalCopies, availableCopies, and isIssued private. I added getters and setters to access and update them safely.
- **Core Operations:** I created the issueToStudent() and returnFromStudent() methods to manage borrowing, returns, and item availability.

### 3. Main Features and Improvements

- **Borrowing Rules for Students and Faculty:** I added two user types: Student and Faculty. Students can borrow up to 3 items for 15 days and Faculty can borrow up to 5 items for 30 days.

#### Fine Calculation and Wikipedia Search

- ✧ Added a late return fine of 20 BDT per day.
- ✧ Added Wikipedia search with email format validation.

**Custom Exception Handling :** I created an Exception package to handle errors clearly. It includes `InvalidInputException`, `ItemNotFoundException`, and `MemberNotFoundException`.

**4. Backend Logic:** I created `FileManager` to save library data and keep it up to date. I also fixed `LibraryService` bugs in item returns and borrow tracking, made title searches case-insensitive, and standardized `memberId` across the models.

**5. GUI and Swing Improvements(Button Style and Visibility):** I updated the button styles in `ViewCatalogPanel` and `UpdateItemPanel` to fix Windows display issues. I also made buttons such as `btnRefresh` visible by setting properties like `opaque` and `borderPainted`.

- **Layout and Tables:** I added a custom purple table header with white text and a category filter for books, magazines, or all items. I also added `AddItemPanel` and `ReturnItemPanel` to the `MainFrame` navigation tabs and adjusted the layout using `GridBagLayout`.

### 6. Git Workflow and Project Status

- **Working Branch:** `feature/library-items`
- **Target Branch:** `main`
- **Status:** The work was reviewed, tested, merged into the target branch, and pushed to the remote repository.

#### Contributor/Member2: Nazia Tazkia

#### **Module: CRUD Operations, File Handling, and Exception Management (Task 2 — CO4)**

##### **1. Building on the Base Structure**

- Puja set up the base `LibraryItems` abstract class with the basic OOP structure. I worked on **extending that class** to add the actual functional logic needed for the CRUD operations — things like issuing, returning, and tracking items.

##### **2. CRUD Operations (Create, Read, Update, Delete)**

- **Create:** I wrote `registerItem()` in `LibraryService` to add new Books/Magazines to the catalog, with checks for title, copy count, and author/publisher before saving.
- **Read:** I wrote `searchByTitle()` (case-insensitive) and `showCatalog()` to look up and display items from the catalog.
- **Update:** I wrote `updateBook()` and `updateMagazine()` to edit existing items, making sure the available copies stay correct even while some copies are issued, and checking for duplicate titles.
- **Delete:** I wrote `removeItem()`, with a safety check so an item can't be deleted if any copy is still borrowed.

### 3. File Handling (Data Persistence)

- Every CRUD action — add, update, issue, return, delete — **saves the data through `FileManager.saveData()`**, so nothing gets lost even after closing the program.
- The catalog also **loads back from file** (`FileManager.loadData()`) whenever the service starts.

### 4. Business Rules and Validation Logic

- I built the **issue process** (`issueItem()` in `LibraryItems`, tied together by `processIssue()` in the service) — checking member ID, member type (Student/Faculty), email format, issue date, available copies, and borrow limits (3 items/15 days for Students, 5 items/30 days for Faculty) before letting an item get issued.
- I built the **return process** (`returnFromMember()`) — checks the return date against the due date and calculates a **late fine of 20 BDT/day** if needed.
- I also added a **borrow-count tracker** (`getBorrowedCountForMember()`) so the system knows how many items a member already has before allowing more.

### 5. Custom Exception Handling

- Used the custom exceptions (`InvalidInputException`, `ItemNotFoundException`, `MemberNotFoundException`) throughout the service layer with **try-catch style validation** — for example, throwing `ItemNotFoundException` if a searched title doesn't exist, or `InvalidInputException` if input data is missing/wrong.

### 6. Git Workflow and Status

- **Working Branch:** `feature/library-service` → merged into `main`
- I reviewed and tested everything before merging, then pushed it to the remote repo.

### Member 3: Thamina islam tenni

#### Module: Magazine Class Implementation, Update Operations, Data Model Improvements, and Testing (Task 1 & Task 2)

##### 1. Magazine Class Implementation

I implemented the Magazine.java class by extending the LibraryItems abstract class.

- **Inheritance:** The Magazine class extends LibraryItems so that it can reuse the common properties and methods of library items.
- **Encapsulation:** The publisher field was kept private, and getter and setter methods were used to access and update it.
- **Polymorphism:** I overrode the getDetails() method to display magazine specific information such as the magazine title, publisher, available copies, and issue status.
- **Constructor:** The magazine title and total number of copies were initialized using the parent class constructor, while the publisher information was initialized in the Magazine constructor.
- **Update Support:** I added the setPublisher(String publisher) method so that publisher information can be modified when a magazine is updated.

##### 2. Update Operations in Library Service

I contributed to the LibraryService update functionality for books and magazines.

- updateBook() changes a book's title, total copies, and author.
- updateMagazine() changes a magazine's title, total copies, and publisher.
- Before updating, the service checks that the item exists and is the correct type.
- It validates that the title is not empty, the copy count is valid, and the title is not already used by another item.
- The total copies cannot be fewer than the copies currently issued.
- Available copies are recalculated as total copies minus issued copies.
- After a successful update, the catalog is saved using FileManager.saveData(catalog).

##### 3. Data Model Improvements

I also contributed setter methods that are required for updating existing library records.

- Added setTitle() in LibraryItems to allow item titles to be updated.
- Added setTotalCopies() and setAvailableCopies() in LibraryItems to maintain correct inventory information.
- Added setMemberId() and setMemberEmail() in LibraryItems to support modification of member-related information.

- Added `setAuthor()` in `Book` so that the author information can be changed during a book update.
- Added `setPublisher()` in `Magazine` so that publisher information can be changed during a magazine update.
- Added `getDueDate()` in `LibraryItems` to provide access to the item's due-date information.

## 9. Conclusion

- **Summary:** The system successfully automates key library operations using standard Java software engineering practices.
- **Objectives Achieved:** All core requirements—GUI navigation, item issue/return, exception handling, and persistent data storage—were fulfilled.
- **Limitations:** Currently uses simple local `.txt` file persistence instead of a relational database system.
- **Future Improvements:** Integrating MySQL/PostgreSQL database backend, adding member authentication, and introducing fine/fee calculation algorithms.